# Pipedrive Integration

See also: [TESTING_GUIDE.md]

## 1. Overview

### 1.1 Description

**Pipedrive Integration** connects **Pipedrive** (sales CRM platform) with **Newo Platform**.

It enables:

- Checking real-time appointment availability by querying Pipedrive Activities
- Booking appointments with automatic contact (person) creation
- Cancelling existing appointments
- Existing customer detection with upcoming booking retrieval
- Automatic customer context injection into agent prompt

This integration is designed for **sales teams and service businesses** who need **automated appointment scheduling and contact management through a conversational AI agent** (voice or chat).

### 1.2 Use Cases

- When a client asks about available slots → Query Pipedrive Activities and calculate free time slots
- When a client wants to book an appointment → Find or create person, then create Activity in Pipedrive
- When a client wants to cancel an appointment → Delete the Activity from Pipedrive
- When a caller's phone is identified → Look up existing person and inject their context + upcoming bookings into prompt
- When a conversation starts → Optionally preload nearest available slots silently

### 1.3 Supported Features

| Feature | Supported | Notes |
|---|---|---|
| Check Availability | ✅ | Calculates free slots from existing Activities |
| Book Appointment | ✅ | Creates Activity with auto person creation |
| Cancel Appointment | ✅ | Deletes Activity from Pipedrive |
| Existing Client Detection | ✅ | By phone, with prompt context injection |
| Upcoming Booking Retrieval | ✅ | Future Activities stored in persona |
| Silent Availability Preload | ✅ | Optional check on conversation start |
| Configurable Slot Duration | ✅ | Default 30 min |
| Configurable Booking Window | ✅ | Days ahead to show (default: 1) |
| Timezone Handling | ✅ | Converts between local and UTC |
| Reschedule Appointment | ❌ | Cancel + rebook as workaround |
| Deal/Pipeline Management | ❌ | Not implemented |

| | | |
|---|---|---|
| Multi-Location Support | ❌ | Single Pipedrive instance per integration |

## 2. Requirements

Before installation:

- Active **Pipedrive** account
- **Pipedrive API Token** (from Personal Preferences → API)
- Pipedrive account URL (e.g., `https://yourcompany.pipedrive.com` )

## 3. How to Setup

### 3.1 Step 1 — Obtain API Token from Pipedrive

1. Log in to your **Pipedrive** account
2. Navigate to **Personal Preferences → API**
3. Copy your **Personal API Token**
4. Store it securely

### 3.2 Step 2 — Connect in Platform

1. Open **Newo → Projects**
2. Set the following attributes in **Builder / Attributes**:

| Attribute | Required | Description |
|---|---|---|
| `pipedrive_api_token` | ✅ | API token for authentication (`x-api-token` header) |
| `pipedrive_base_api_url` | ❌ | Pipedrive account URL (default: `https://newo.pipedrive.com`) |
| `pipedrive_duration` | ❌ | Appointment duration in minutes (default: `30`) |
| `pipedrive_show_for_days` | ❌ | Days ahead for availability (default: `1`) |
| `pipedrive_slots_available` | ❌ | Enable availability checking (default: `True`) |
| `pipedrive_booking_available` | ❌ | Enable booking capability (default: `True`) |
| `pipedrive_cancelation_available` | ❌ | Enable cancellation capability (default: `True`) |
| `pipedrive_check_availability_on_conversation_start` | ❌ | Pre-load slots on session start (default: `False`) |

3. Click **Save + Publish All**
4. On publish, the SetupFlow automatically:
    - Validates the API token
    - Creates the HTTP connector for Pipedrive

- Injects booking and availability payload schemas for the agent

# 4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

## 4.1 How the Integration Works

The integration uses Pipedrive **Activities** as appointments and **Persons** as contacts.

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in Pipedrive

**Available Triggers**

| Trigger | Event | Description |
|---|---|---|
| **Check Availability** | `get_available_slots` | When the agent needs to look up open slots |
| **Book Appointment** | `book_slot` | When the agent confirms a booking |
| **Cancel Appointment** | `convo_agent_cancel_booking` | When the agent processes a cancellation |
| **Phone Identified** | `define_user_phone_number` | When the caller's phone is detected |
| **Session Start** | `session_started` | Optional silent availability preload |
| **Setup** | `project_publish_finish` | Initializes integration on deploy |

**Available Actions**

- **Search Person** — Find contact by phone (GET `/v1/persons/search`)
- **Create Person** — New contact with name and phone (POST `/api/v2/persons`)
- **Get Activities** — Fetch booked appointments (GET `/api/v1/activities`)
- **Get Person Activities** — Fetch contact's bookings (GET `/api/v2/activities?person_id={id}`)
- **Create Activity** — Book an appointment (POST `/v1/activities`)
- **Delete Activity** — Cancel an appointment (DELETE `/api/v2/activities/{id}`)

## 4.2 Architecture Diagrams

**Overall Sequence**

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as Pipedrive Integration
    participant API as Pipedrive API

    Note over I: Setup (on publish)
    A->>I: project_publish_finish
    I->>I: Create connector, inject schemas
    I->>A: Tools configured

    Note over U,API: Existing Client Detection
    A->>I: define_user_phone_number (phone)
```

```
    I->>API: GET /v1/persons/search (phone)
    API-->>I: Person data
    I->>A: Inject ExistingCustomerInfo into prompt
    I->>API: GET /api/v2/activities?person_id=(id)
    API-->>I: Future bookings
    I->>A: Store bookings in persona

    Note over U,API: Availability Check
    U->>A: "Any slots tomorrow?"
    A->>I: get_available_slots (date, time)
    I->>API: GET /api/v1/activities (date range, done=false)
    API-->>I: Booked activities
    I->>I: Calculate free slots (subtract booked, slice by duration)
    I->>A: result_success (availability[])
    A->>U: "Here are the available times..."

    Note over U,API: Booking
    U->>A: "Book 2:00 PM please"
    A->>I: book_slot (customerInfo, dateTime)
    I->>API: GET /v1/persons/search (phone)
    alt Person not found
        I->>API: POST /api/v2/persons (name, phone)
        API-->>I: person_id
    end
    I->>API: POST /v1/activities (due_date, due_time, duration, person_id)
    API-->>I: activity_id
    I->>A: result_success (booking_id)
    A->>U: "Your appointment is confirmed!"

    Note over U,API: Cancellation
    U->>A: "Cancel my appointment"
    A->>I: convo_agent_cancel_booking (booking_id)
    I->>API: DELETE /api/v2/activities/(booking_id)
    API-->>I: Confirmation
    I->>A: result_success (cancelled)
    A->>U: "Your appointment has been cancelled"
```

**AvailabilityFlow**

```
flowchart TD
    E["get_available_slots<br>or session_started"] --> GATE{"Slot
check<br>enabled?"}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| RANGE["Calculate date range:<br>date → date + show_for_days"]
    RANGE --> API["GET /api/v1/activities<br>{updated_since, updated_until,
<br>done=false}"]
    API --> OK{"HTTP 200?"}
    OK -->|"No"| ERR["ResultError"]
    OK -->|"Yes"| CALC["Parse booked activities<br>Create duration-based
slots<br>Subtract overlapping"]
```

```
    CALC --> TZ["Convert UTC<br>to local timezone"]
    TZ --> OUT["result_success<br>{availability[]}"]
```

**BookingFlow**

```
flowchart TD
    E["book_slot"] --> GATE{"Booking<br>enabled?"}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| SEARCH["GET /v1/persons/search<br>{phone, fields: phone}"]
    SEARCH --> FOUND{"Person<br>found?"}
    FOUND -->|"Yes"| PID["Use person_id"]
    FOUND -->|"No"| CREATE["POST /api/v2/persons<br>{name, phones}"]
    CREATE --> PID
    PID --> UTC["Convert time to UTC"]
    UTC --> API["POST /v1/activities<br>{due_date, due_time, duration,<br>subject,
type: meeting,<br>person_id}"]
    API --> OK{"HTTP 200?"}
    OK -->|"No"| ERR["ResultError"]
    OK -->|"Yes"| OUT["result_success<br>{booking_id: activity_id}"]
```

**CancellationFlow**

```
flowchart TD
    E["convo_agent_cancel_booking"] --> GATE{"Cancellation<br>enabled?"}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| VAL{"booking_id<br>present?"}
    VAL -->|"No"| ERR["ResultError"]
    VAL -->|"Yes"| API["DELETE /api/v2/activities<br>/{booking_id}"]
    API --> OK{"Success?"}
    OK -->|"No"| ERR
    OK -->|"Yes"| OUT["result_success<br>{cancel_booking}"]
```

**ExistingClientFlow**

```
flowchart TD
    E["define_user_phone_number"] --> SEARCH["GET /v1/persons/search<br>{phone}"]
    SEARCH --> FOUND{"Person<br>found?"}
    FOUND -->|"No"| DONE["No action"]
    FOUND -->|"Yes"| INJECT["Inject ExistingCustomerInfo<br>into agent prompt<br>
{name, email}"]
    INJECT --> APPTS["GET /api/v2/activities<br>?person_id={id}"]
    APPTS --> FILTER["Filter future<br>activities only"]
    FILTER --> SAVE["Store in persona<br>attribute: bookings<br>(deduplicated by
booking_id)"]
```

### 4.3 Where to Test

Testing can be done through the **Newo conversation interface** (chat or voice channel). BookingFlow and CancellationFlow have dedicated test skills for connectivity validation.

### 4.4 Testing and Verification

**To test the integration:**

1. **Setup**: Publish the project and verify schemas are injected
2. **Existing Client**: Start a conversation from a known phone — verify customer info injected into prompt
3. **Availability**: Ask "What slots are available tomorrow?" — verify free slots calculated correctly
4. **Booking**: Complete a booking — verify Activity created in Pipedrive with correct person link
5. **Cancellation**: Cancel a booking — verify Activity deleted from Pipedrive

**If no action occurs:**

- Ensure `pipedrive_api_token` is set correctly
- Verify `pipedrive_base_api_url` points to your Pipedrive account (not the default)
- Confirm the integration was re-published after changes
- Check feature flags are enabled ( `pipedrive_slots_available` , `pipedrive_booking_available` , `pipedrive_cancelation_available` )
- Verify timezone is set correctly via `pipedrive_timezone`

> *Note: This integration performs real operations in Pipedrive. Activities and persons created or deleted through the agent are reflected in the live CRM system.*

## 5. FAQ

**Q: Does the integration import historical data?** A: No. Only Activities created after activation are managed. Existing client detection retrieves future Activities for context only.

**Q: Can I connect multiple Pipedrive accounts?** A: Each integration instance supports one API token. For multiple accounts, create separate integration instances.

**Q: How does availability checking work?** A: The integration fetches incomplete Activities (done=false) within the date range and calculates free time slots by finding gaps between booked activities. Slot duration is configurable via `pipedrive_duration` (default: 30 minutes).

**Q: What happens if a contact already exists?** A: The integration searches persons by phone number. If found, the existing person is used for booking. If not, a new person is created with name and phone.

**Q: Why is the default booking window only 1 day?** A: `pipedrive_show_for_days` defaults to 1 for performance. Increase it to show more days of availability (e.g., 7 for a week).

**Q: How are phone numbers handled?** A: Phone numbers are expected in E.164 format. The leading `+` is stripped for Pipedrive search queries. Search matches against all phone numbers associated with a person.

**Q: What Pipedrive entity represents an appointment?** A: Appointments are stored as Pipedrive **Activities** of type "meeting". The Activity subject includes the customer name and formatted time (e.g., "John Doe — Sep 28 at 2:00 PM").